❯ This is a Pygame platformer lab. Read README.md, main.py and everything in
  game/. Install the requirements with pip. Then give me a short summary of how
  the code is structured and where the collision logic lives. Don't change any
  code yet. Don't add a Co-Authored-By line to any commit you make. Use the
  existing git user.name and user.email as-is.

  Read 5 files, listed 1 directory, ran 2 shell commands (ctrl+o to expand)

● I read the README, main.py and everything in game/, and installed the
  requirements (pygame 2.6.1). I changed no code. I also made no commits, so
  your instructions about the Co-Authored-By line and git user settings didn't
  come into play.

  Structure

  The project is in /Users/harish/44_simple-platformer.

  - main.py opens an 800×500 window and creates a GameEngine. Its loop runs at
    60 FPS. Each frame it fills the sky colour, passes events to
    engine.handle_event, then calls handle_input(), update() and render(), and
    flips the display.
  - game/game_engine.py is where almost everything happens:
    - Setup: GameEngine holds the gravity value (0.6), the player, four
      hand-placed platforms, one hazard, the goal x-position (740), the score
      and the game_over flag.
    - Input: handle_event triggers jumps on keydown. handle_input polls held
      keys to set horizontal velocity.
    - Frame step: update() applies gravity and movement, checks collisions, then
      checks game over and the goal.
    - Drawing: render() draws the platforms, hazard, goal line, player and
      score. It also prints "Game over" to the console, which is what Task 2
      asks you to replace.
  - game/player.py holds the player's position, velocity, speed, jump strength
    and on_ground flag. It has rect() and jump(), which only works when
    on_ground is true.

```
  - game/platform.py and game/hazard.py are nearly identical. Each
holds x, y,
    width and height (default 14) and returns a pygame.Rect.

  Where the collision logic lives

  All of it is in GameEngine.update() in game/game_engine.py.

  - Platforms (lines 68–74 (game/game_engine.py#L68-L74)):
    - The player moves by vy, then on_ground is reset.
    - For each platform, if colliderect is true and vy >= 0, the
player is
      snapped onto the platform top and vy is zeroed.
  - Hazards (lines 76–79 (game/game_engine.py#L76-L79)): an overlap
sets
    game_over.
  - Falling off the screen (line 81 (game/game_engine.py#L81)): y >
height sets
    game_over.

  The Task 1 bug. Gravity has no terminal-velocity cap, and
collision only
  compares the player's rect after the move. After a long fall, such
as off the
  raised platform at x=420, one frame's vy can exceed the platform's
14px
  thickness plus the player's height. The player then goes from
above the
  platform to below it without the rects overlapping, and falls
through. The
  comments at lines 58–67 describe this.

  The fix would be a swept check that compares the player's previous
bottom edge
  to the platform top, and/or a cap on vy. Say when you want me to
make
  changes.

❯ Task 1: The player sometimes falls straight through a platform
instead of
  landing on it, especially after a long fall. Fix collision in
  GameEngine.update() so landings register reliably at any fall
speed: compare
  the player's previous bottom edge against each platform's top
(swept check),
  and also cap the maximum fall speed. Briefly explain the change.
Then commit
  with the message "Task 1: Fix collision tunneling through
platforms". Don't
  touch anything else.

● Update(game/game_engine.py)
  ⎿  Added 13 lines, removed 11 lines
      55          self.player.vy += self.gravity
```

```
      56          self.player.x = max(0, self.player.x +
self.player.vx)
      57
      58 -        # NOTE: gravity has no terminal-velocity cap, so
vertic
         -al speed
      59 -        # keeps growing the longer the player falls.
Collision
         -is only
      60 -        # checked against the player's rect *after* it has
alre
         -ady
      61 -        # moved for the frame - there's no check for
whether th
         -e
      62 -        # player's path crossed a platform along the way.
After
         - a
      63 -        # long enough fall (e.g. off the elevated middle
platfo
         -rm),
      64 -        # a single frame's movement can carry the player
from j
         -ust
      65 -        # above a platform to just below it without the
two rec
         -ts
      66 -        # ever overlapping, so the platform is skipped
entirely
         - and
      67 -        # the player falls straight through. See Task 1 in
the
         -README.
      58 +        # Cap the fall speed so a single frame can't move
the p
         +layer far.
      59 +        self.player.vy = min(self.player.vy,
MAX_FALL_SPEED)
      60 +
      61 +        prev_bottom = self.player.y + self.player.height
      62          self.player.y += self.player.vy
      63 +        new_bottom = self.player.y + self.player.height
      64          self.player.on_ground = False
      65          for platform in self.platforms:
      71 -            if
self.player.rect().colliderect(platform.rect())
         -and self.player.vy >= 0:
      66 +            # Swept check: land if the player's bottom
edge wen
         +t from
      67 +            # at/above the platform top to at/below it
this fra
         +me, even
      68 +            # if the two rects never overlap at the end of
```

```
the
         +frame.
      69 +            overlaps_x = (self.player.x < platform.x +
platform
         +.width
      70 +                          and self.player.x +
self.player.width
         + > platform.x)
      71 +            crossed_top = prev_bottom <= platform.y <=
new_bott
         +om
      72 +            if overlaps_x and crossed_top and
self.player.vy >=
         + 0:
      73                  self.player.y = platform.y -
self.player.height
      74                  self.player.vy = 0
      75                  self.player.on_ground = True
      76 +                new_bottom = platform.y
      77
      78          for hazard in self.hazards:
      79              if
self.player.rect().colliderect(hazard.rect()):
```

● Update(game/game_engine.py)
  ⎿ Added 2 lines

```
      10  RED = (220, 60, 60)
      11  GREEN = (0, 200, 0)
      12
      13 +MAX_FALL_SPEED = 15
      14 +
      15  class GameEngine:
      16      def __init__(self, width, height):
      17          self.width = width
```

  Ran 1 shell command (ctrl+o to expand)

● I fixed the tunneling in GameEngine.update() in game/
game_engine.py and
  committed it as "Task 1: Fix collision tunneling through
platforms". I didn't
  touch any other files.

  What changed
  - Fall speed cap: a new MAX_FALL_SPEED = 15 constant. vy is
clamped to it each
    frame, so one frame can't move the player very far.
  - Swept landing check: the player's bottom edge is recorded before
the move.
    The player lands if that edge was at or above the platform top
and is at or
    below it after the move, and the player overlaps the platform
horizontally.
    This works even when the player's rect and the platform's rect

never overlap
    at the end of the frame. The player is then snapped to the platform top, vy
    is set to 0 and on_ground is set to true.
  - Comment: I replaced the old NOTE comment that described the bug.

  Testing
  I dropped the player onto the elevated platform at x=450 in a headless run. It
  landed with on_ground true and game_over false, so the player did not fall
  through. I didn't play the game by hand.

  Commit
  - I didn't add a Co-Authored-By line, as you asked.
  - The commit used the existing git identity, Harish Rameshkumar
    harishindiahosur@gmail.com.
  - I staged only game_engine.py. The game/__pycache__/*.pyc files show as
    untracked and were left out.

❯ Task 2: Add a Game Over screen that shows the final score when the player
  falls off the bottom of the screen or touches the hazard. It should be drawn
  in the Pygame window, wait for key input, and not print to the console. Keep
  the existing code style. Then commit with the message "Task 2: Add Game Over
  screen with final score". Don't add a Co-Authored-By line.

  Ran 1 shell command (ctrl+o to expand)

● The on-disk change is just my own edit, so I'll go ahead and commit.

  Ran 1 shell command (ctrl+o to expand)

● I added the Game Over screen in game/game_engine.py and committed it as "Task
  2: Add Game Over screen with final score". There's no Co-Authored-By line.

  I tested it headless: the player fell into the gap, the screen came up, and a
  keypress posted the quit event. I haven't played it by hand.

  - What it shows: a dark overlay over the frozen scene with "Game Over", "Final
    Score: N" and "Press any key to exit". It's drawn in the Pygame window, and
    the console print and the _game_over_logged flag are gone.
  - When it appears: update() already set game_over for a hazard hit or a fall

    off the bottom, so the screen covers both.
  - Input: once the game is over, handle_event ignores the game controls. Any
    key press then posts a QUIT event, which ends the existing loop in main.py.
  - Added delay: keys are ignored for the first 30 frames (about half a second)
    after death. Without that, a key you were holding when you died would close
    the window at once. The constant is GAME_OVER_INPUT_DELAY.
  - Task 3: the prompt says "exit" for now. When you add the replay and
    difficulty choice, that key handler is where it goes.

  I committed only game_engine.py, so the __pycache__ files are still untracked.

❯ Task 3: On the Game Over screen, replace the "press any key to exit"
  behavior. Let the player press 1, 2 or 3 to replay on Easy, Medium or Hard
  (each changes gravity and jump strength; Hard should have stronger gravity
  and a weaker jump, Easy the opposite), or press Esc or Q to exit. Show these
  options on the screen. On replay, reset the score, the player state and the
  game_over flag, and apply the chosen difficulty. Keep the game-over input
  delay. Then commit with the message "Task 3: Add replay with difficulty
  selection". Don't add a Co-Authored-By line.

  Ran 1 shell command (ctrl+o to expand)

● Task 3 is done and committed as "Task 3: Add replay with difficulty
  selection", with no Co-Authored-By line. All changes are in
  game/game_engine.py. I tested it headless but haven't played it by hand.

  - Difficulties: a DIFFICULTIES table maps keys 1, 2 and 3 to these values:

Key	Level	Gravity	Jump strength
1	Easy	0.5	-13
2	Medium	0.6	-12
3	Hard	0.7	-11
    Medium is the original game. I picked the Easy and Hard numbers myself, so
    adjust them if you want a different feel. The 60px raised platform is still
    reachable on Hard.

  - Replay: a new restart() method applies the chosen gravity and
jump strength.
    It builds a fresh player at the start position and resets the
score,
    game_over and the frame counter.
  - Exit: Esc or Q posts the quit event.
  - Input delay: keys are still ignored for the first 30 frames
after death.
    Other keys do nothing once that passes.
  - Screen: it shows "Game Over", the final score, "Replay: 1 - Easy
2 -
    Medium   3 - Hard" and "Esc / Q - Exit".

  In the headless test, pressing 3 after dying reset the score to 0
and applied
  gravity 0.7 and jump -11. Q posted the quit event.

  I committed only game_engine.py; the __pycache__ files are still
untracked.

❯ Task 4: Add sound effects for jumping, reaching the goal, and
dying. Generate
  the sounds in code (synthesized tones with pygame.mixer and numpy
or the
  array module), so there are no external audio files. Play the jump
sound on
  each successful jump, the goal sound when the player reaches the
goal, and
  the death sound once when the game over screen starts. If the
audio device
  fails to initialise, the game must keep running silently. Check
numpy is in
  requirements.txt if you use it. Then commit with the message "Task
4: Add
  sound effects". Don't add a Co-Authored-By line.

  Ran 2 shell commands (ctrl+o to expand)

● Task 4 is done and committed as "Task 4: Add sound effects", with
no
  Co-Authored-By line. I tested it headless, with a dummy audio
driver and with
  a deliberately invalid one. I haven't heard the sounds on a real
speaker, so
  check how they sound.

  Sounds
  - No new dependency: I used the standard-library array module
rather than
    numpy, so requirements.txt still only lists pygame.
  - New file: game/sound.py synthesizes sine tones with a fade-out
and builds
    them as pygame.mixer.Sound objects. Nothing is loaded from disk.
  - Tones: jump is a quick rising sweep. Goal is a three-note rising

```
arpeggio.
    Death is a falling sweep that lasts half a second.
  - Fallback: if the mixer won't start or gives an unexpected sample
format,
    every sound becomes None and play() does nothing. With an
invalid audio
    driver the game kept running silently.

  Wiring in game/game_engine.py
  - Jump: it plays only when the player is on the ground, so mid-air
presses
    stay silent. In the test, one grounded press and one mid-air
press made one
    sound.
  - Goal: it plays when the score increments.
  - Death: a new die() method sets game_over and plays the death
sound. The
    hazard hit and the fall off the bottom both call it. The game-
over branch of
    update() returns early, so the sound plays once, when the game
over screen
    starts.

  I committed game/sound.py and game_engine.py only. The __pycache__
files are
  still untracked.

❯ Create a Lab-4 folder in the repo. Copy main.py, requirements.txt
and the
  game/ folder into it, excluding __pycache__. Move ~/Desktop/
before.mov and
  ~/Desktop/after.mov into Lab-4. Commit as "Add Lab-4 code and
videos". Then
  show me the output of: git log --format='%an <%ae> | %s'. Do not
push. Don't
  add a Co-Authored-By line.

  Listed 1 directory, ran 1 shell command (ctrl+o to expand)

● I created Lab-4/ and committed it as "Add Lab-4 code and videos".
I did not
  push, and the commit has no Co-Authored-By line.

  Lab-4/ holds main.py, requirements.txt, the game/ folder (without
  __pycache__), and the two videos. I moved before.mov and after.mov
off the
  Desktop, so they're no longer there.

  Output of git log --format='%an <%ae> | %s':

  Harish Rameshkumar <harishindiahosur@gmail.com> | Add Lab-4 code
and videos
  Harish Rameshkumar <harishindiahosur@gmail.com> | Task 4: Add
sound effects
```

```
  Harish Rameshkumar <harishindiahosur@gmail.com> | Task 3: Add
replay with
  difficulty selection
  Harish Rameshkumar <harishindiahosur@gmail.com> | Task 2: Add Game
Over screen
  with final score
  Harish Rameshkumar <harishindiahosur@gmail.com> | Add .gitignore
for pycache
  Harish Rameshkumar <harishindiahosur@gmail.com> | Task 1: Fix
collision
  tunneling through platforms
  kwazzy-coder <samrudhanarendra2005@gmail.com> | Initial Simple
Platformer Main
  starter

  There's an "Add .gitignore for pycache" commit that I didn't make
in this
  session. It sits between Task 1 and Task 2, so something else
added it during
  the session. The only commit by someone else is the original
starter commit
  from kwazzy-coder.

  The two videos total about 88 MB, so the repo is now much larger.
GitHub
  rejects individual files over 100 MB, and these are under that,
but you may
  want to know before pushing.
```